



**Софийски университет „Св. Кл.
Охридски“**

Факултет по математика и информатика

*Бакалавърска програма
„Анализ на данни“*

Дисциплина: Езици и технологии на метаданни

Летен семестър, 2025/2026 год.

**Тема № 9: „Инструмент за събиране на
метаданни (Metadata Harvesting Tool)“**

Курсов проект

Автори:

Нели Лазарова, фак. номер 9M10200039

Никола Джалев, фак. номер 4M10200052

юни, 2026 г.

София

Съдържание

1	Въведение	3
2	Анализ на решението.....	3
2.1	Работен процес	3
2.2	Тип и представяне на съдържанието	4
2.3	Използване на изкуствен интелект (ИИ) за разработването на проекта	4
3	Дизайн	4
4	Тестване	4
5	Заклучение и възможно бъдещо развитие	5
6	Разпределение на работата.....	5
7	Използвани литературни източници и Уеб сайтове.....	5
8	Апендикс.....	5

За разработването на курсов проект по „Езици и технологии на метаданни“ трябва да използвате настоящият шаблон за оформление на документация на курсов проект. За всяка секция е указано задължително съдържание и препоръчителен обем в страници. Всяка секция трябва да бъде внимателно и детайлно попълнена. Използването на шаблона и спазването на описаните правила в него са задължителни.

1 Въведение

В съвременната дигитална среда се работи с големи количества документи и онлайн ресурси — PDF файлове, Excel таблици, уеб страници, научни публикации и API източници. За по-лесното им търсене, класифициране и обработка е необходимо да се събират **метаданни**, тоест информация за самите ресурси — име, автор, дата на създаване, дата на промяна, формат, размер, ключови думи, DOI и др.

Основният проблем, който проектът решава, е, че различните типове източници съхраняват метаданните по различен начин. PDF документите имат един вид вградена информация, Excel файловете — друга, а научните публикации в интернет често предоставят данни чрез външни API услуги. Това прави ръчното събиране на метаданни бавно, неудобно и податливо на грешки.

Целта на проекта е разработване на софтуерен инструмент за автоматично събиране на метаданни от различни източници и представянето им в унифициран формат. Системата ще използва отделни модули за обработка на различни типове входни данни — например PDF файлове, електронни таблици, URL адреси и научни API източници като CrossRef. След извличането информацията ще спрямо обща структура - Dublin Core, така че резултатите да могат да се показват и експортират лесно.

Останалата част от документа разглежда основните понятия, свързани с метаданните, архитектурата на системата, използваните технологии, начина на реализация и тестването на инструмента с различни типове източници.

2 Анализ на решението

2.1 Работен процес

В началния етап на разработката на проекта беше необходимо да се избере подходящ метод за извличане на метаданни от различни типове източници. Разгледани бяха няколко възможности за обработка на PDF документи, Excel файлове, URL адреси към научни публикации и външни API източници. Основното предизвикателство е, че всеки тип ресурс има различна структура и изисква различен начин на обработка. Например PDF файлът се нуждае от специализиран парсер за извличане на вградени свойства и брой страници, Excel файлът изисква обработка на работни листове и свойства на таблицата, а

URL адрес към научна публикация може да изисква извличане на HTML метаданни или заявка към външен API.

Първоначалният подход беше да се дефинират предварително всички възможни типове входни данни и за всеки от тях ръчно да се избере конкретен метод за обработка. В процеса на анализ обаче стана ясно, че това може да направи системата по-трудна за разширяване, тъй като при добавяне на нов тип файл или източник ще трябва да се променя логиката за разпознаване и избор на подходящ инструмент.

Поради тази причина беше избрано използването на LLM компонент като **оркестратор на работния процес**. LLM не се използва като основен инструмент за директно извличане на метаданни, а като модул, който анализира подадения вход, разпознава типа на ресурса и избира кой специализиран скрипт или extractor трябва да бъде извикан. По този начин системата остава гъвкава и може по-лесно да работи с различни видове файлове и онлайн източници.

Например, ако потребителят подаде PDF документ, LLM оркестраторът насочва обработката към PDF extractor. Ако входът е Excel файл, се извиква spreadsheet extractor. При URL адрес или DOI се използва модул за извличане на уеб метаданни или заявка към външен API, например CrossRef. След като съответният скрипт извлече суровите метаданни, LLM подпомага тяхното нормализиране към обща структура, съобразена със стандарт като Dublin Core, което отговаря на целта на проекта за представяне на събраните метаданни в унифициран формат.



Диаграмата е генерира с ChatGPT.

За реализацията на отделните обработващи модули е избран езикът Python, тъй като той предлага голям набор от готови библиотеки за работа с различни типове файлове, уеб заявки и API услуги. Освен това Python кодът е лесен за четене, поддръжка и разширяване, което го прави подходящ за проект, в който отделните extractors могат да бъдат добавяни

постепенно. Документацията в repository-то ще бъде написана на английски език, тъй като английският е по-лесен за интерпретиране от AI agents и LLM модели. Това улеснява използването на такива инструменти при генериране, анализ и поддръжка на кода, както и при бъдещо разширяване на системата с нови функционалности.

2.2 Използване на изкуствен интелект (ИИ) за разработването на проекта

При разработката на проекта беше използван изкуствен интелект като помощен инструмент както за планиране на архитектурата, така и за реализацията и документирането на отделните компоненти. Основната идея на проекта е инструмент за събиране на метаданни, който приема различни входни ресурси, извлича сурови метаданни и ги представя в унифициран формат, съобразен със стандарти като Dublin Core. В началния етап AI беше използван основно занасоки и структура. Чрез него бяха сравнени различни подходи за обработка на PDF документи, URL адреси, DOI идентификатори и външни API източници. Това помогна да се избере архитектура, при която LLM не извлича директно всички метаданни, а действа като оркестратор (диригент). Неговата задача е да разпознае типа на входния ресурс и да насочи обработката към подходящия extractor. В текущото repository процесът е описан като поток от входен ресурс към LLM orchestrator, след което към PDF, Excel, URL, DOI или API extractor, сурови метаданни, нормализиране и унифициран изход.

За основен език беше избран Python, тъй като е подходящ за бързо изграждане на отделни скриптове за обработка на файлове, уеб заявки и API услуги. В проекта се използват библиотеки като Streamlit, google-generativeai, PyMuPDF, requests, BeautifulSoup, Pydantic и python-dotenv.

AI беше полезен и при нормализирането на данните. Тъй като различните източници могат да връщат сходна информация с различни имена на полета, LLM може да помогне при преобразуването им към обща схема. Например полета като author, creator и authors могат да бъдат обединени в едно поле за създател, а title, name и paperTitle — в поле за заглавие. Това подпомага целта на проекта резултатите да бъдат представени в унифициран формат.

Изкуственият интелект беше използван и при отстраняване на неизправности и разрешаване на проблеми. При грешки в обработката на входните ресурси AI помогна за анализ на възможните причини, като например неправилен формат на DOI, липсващи metadata тагове, празни PDF metadata полета или проблеми при заявки към външни API услуги. Това улесни добавянето на проверки, обработка на грешки и по-ясна структура на кода.

AI подпомогна и процеса на обратна връзка и итерация. Архитектурата на проекта беше подобрявана постепенно — първо чрез идея за отделни extractors, след това чрез добавяне на LLM оркестратор, а накрая чрез нормализиране към обща схема. Така проектът се развива по модулен начин и може по-лесно да бъде разширен с нови типове файлове или нови източници на метаданни.

Искусственият интелект беше използван и при документирането на проекта. Документацията в repository-то е на английски език, тъй като английският е по-лесен за интерпретиране от AI agents и LLM модели. Това улеснява автоматичното генериране на обяснения, анализ на кода, поддръжка на проекта и бъдещо разширяване на системата. По този начин AI не замества разработчика, а служи като помощен инструмент за структуриране, генериране на идеи, писане на примерен код, отстраняване на проблеми и документиране.

3 Дизайн

Тази секция представя дизайна на решението, използваните технологии, архитектурата на системата и начина, по който отделните компоненти взаимодействат помежду си. Основната цел на проекта е разработване на софтуерен инструмент за събиране на метаданни от различни източници и представянето им в унифициран формат. Решението е проектирано така, че да може да обработва различни типове входни ресурси, като PDF документи, URL адреси, DOI идентификатори и външни API източници. В началния етап на проектиране беше избран модулен подход. Вместо всички типове данни да се обработват в един общ скрипт, системата е разделена на отделни компоненти, всеки от които има ясно определена отговорност. Това прави проекта по-лесен за разбиране, поддръжка и бъдещо разширяване. Ако в бъдеще се добави нов тип входен ресурс, например Excel файл, може да се създаде нов extractor модул, без да се променя цялата архитектура.

За реализацията на проекта е избран езикът Python. Причината е, че Python предлага богата екосистема от библиотеки за обработка на файлове, уеб заявки, API услуги и структурирани данни. Освен това синтаксисът му е лесен за четене и разбиране, което го прави подходящ за работа с AI agents и LLM модели. Потребителският интерфейс е реализиран чрез Streamlit. Тази библиотека позволява бързо изграждане на уеб интерфейс, чрез който потребителят може да избере тип входен ресурс, да въведе URL или DOI, да качи PDF файл и да види резултата от обработката.

За работа с PDF файлове се използва PyMuPDF. Тази библиотека позволява извличане на вградени метаданни от PDF документи, като заглавие, автор, дата на създаване, дата на последна промяна, subject и keywords, когато такива са налични. За обработка на URL адреси се използват requests и BeautifulSoup. Requests изпраща HTTP заявка към подадената страница, а BeautifulSoup се използва за анализ на HTML съдържанието и извличане на metadata тагове. За DOI идентификатори се използва CrossRef API. Чрез него системата може да получи библиографски метаданни за научни публикации, като заглавие, автори, издател, дата на публикуване, език, URL и други налични полета. За връзка с LLM модела се използва библиотеката google-generativeai. Тя позволява системата да използва Gemini модел за оркестрация, обогатяване и преобразуване на суровите метаданни към общатрукура. За дефиниране и валидиране на крайния модел се използва Pydantic. Чрез него е създаден

Dublin Core модел, който гарантира, че изходните данни имат предвидима и структурирана форма.

3.2 Архитектура на приложението

Приложението е разделено на няколко основни файла и модула. Основният файл е `app.py`, който управлява потребителския интерфейс и свързва останалите части на системата. В него потребителят избира типа входен ресурс, подава данните и стартира процеса по извличане на метаданни. Файлът `orchestrator.py` съдържа логиката, свързана с LLM компонента. Неговата роля е да подпомага разпознаването на входния ресурс и преобразуването на извлечените сурови метаданни към по-структуриран вид. Папката `extractors` съдържа отделните модули за извличане на метаданни. Всеки `extractor` е специализиран за конкретен тип ресурс. Например `pdf_extractor.py` обработва PDF файлове, `url_extractor.py` обработва URL адреси, а `doi_extractor.py` обработва DOI идентификатори чрез CrossRef API. Файлът `normalizer.py` отговаря за преобразуването на обогатените метаданни към Dublin Core модела. Файлът `models.py` съдържа дефиницията на Dublin Core структурата, а `serializers.py` отговаря за преобразуването на крайния резултат в JSON и JSON-LD формат.

4 Тестване (2-3 стр.)

За тестването бяха използвани няколко различни типа входни данни, за да се провери дали системата работи коректно при различни сценарии. Основните типове данни включват PDF файлове, URL адреси към уеб страници и научни публикации, както и DOI идентификатори, които се използват за извличане на метаданни чрез външен API източник. Първият тип тестови данни бяха PDF документи. При тях беше проверено дали приложението може да приеме качен файл и да извлече наличните вградени метаданни. Такива метаданни могат да бъдат заглавие, автор, дата на създаване, дата на последна промяна, ключови думи и други свойства, ако са налични в самия PDF файл. При някои PDF файлове част от тези полета липсваха, което също беше полезно за тестване, защото системата трябва да може да работи и с непълни метаданни. Вторият тип тестови данни бяха URL адреси. При тях беше проверено дали системата може да достъпи уеб страница, да прочете HTML съдържанието ѝ и да извлече `metadata` тагове. Тествани бяха страници с налични `meta` тагове, както и страници, при които част от `metadata` информацията липсва. Това позволи да се провери дали URL `extractor`-ът връща наличната информация без да прекъсва изпълнението при липсващи полета. Третият тип тестови данни бяха DOI идентификатори. При тях беше проверено дали системата може да изпрати заявка към CrossRef API и да получи библиографски метаданни за научна публикация. Тестването с DOI е важно, защото демонстрира работа с външен API източник, което е една от основните цели на проекта. Допълнително бяха използвани и невалидни входни данни. Например беше въведен грешен DOI, празен URL адрес и липсващ PDF файл. Целта на тези тестове беше да се провери дали приложението показва подходящо съобщение за грешка и не прекъсва неочаквано.

5 Заключение и възможно бъдещо развитие

Проектът беше завършен успешно, тестовите се покриват правилно. Бъдещи стъпки за развитие могат да са използване на по-добър LLM модел - например opus 4.7, gpt 5.5 и тн, добавяне на [SKILL.md](#) файлове и агенти, за по-автомно вземане на решения и класифициране на типа файлове и метаданни.

Чрез използването на по-добри модели тренирани с повече параметри, изграденият инструмент ще може да разпознава много повече видове файлове и съответно техните метаданни.

Друга полезна посока може да е да се направи сравнение с подобни имплементации на заданието.

6 Разпределение на работата

Нели Лазарова + Никола Джалев - архитектура на проекта (код). Планиране на нужните стъпки за изпълнение. Изготвяне на спецификации за AI агенти (.md files), които позволява по-ефективно изпълнение на работа и заявки. Разписване на планове и промптове за AI. Проучване на текущите използвани методи и инструменти.

Нели Лазарова - избор на LLM модел за оркестратор и агенти използвани в разработката на софтуера (claude code, open code), управление на Google AI панел и ресурси.

Никола Джалев - UI, тестове и верификация. GitHub setup и контрол на версиите. Проучване и избор за възможни FE Python решения. Писане на код за някои функционалности.

7 Използвани литературни източници и Уеб сайтове

1. <https://streamlit.io/>
2. <https://aistudio.google.com/>
3. <https://cursor.com/docs>
4. <https://code.claude.com/docs/en/overview>
5. <https://opencode.ai/docs/>
6. https://en.wikipedia.org/wiki/Dublin_Core
7. <https://www.dublincore.org/>